

2a. For this assignment, I used Claude and ChatGPT.

2b. I accessed both Claude and ChatGPT on my laptop.

2c. Here is the prompt from both programs: "Write a C program that picks a secret number between 1 and 10 (use a fixed number like 7). The user gets up to 3 tries to guess it. Use printf to prompt and scanf to read input, a loop for the guesses, and if-else to say too high/too low/correct. Stop early if they guess right, and print a final win/lose message. Here's an example of the expected output:

Guess a number between 1 and 10.

Attempt 1/3. Enter your guess: 5

Too low! Try again.

Attempt 2/3. Enter your guess: 8

Too high! Try again.

Attempt 3/3. Enter your guess: 7

Correct! You win!

```

2d. (AI Programs)

(I added these here just in case but they're in the main folder as well :D )

(Claude)

```
#include <stdio.h>

int main() {
 int secret = 7;
 int guess;
 int attempts = 3;

 printf("Guess a number between 1 and 10.\n");

 for (int i = 1; i <= attempts; i++) {
 printf("Attempt %d/%d. Enter your guess: ", i, attempts);
 scanf("%d", &guess);

 if (guess == secret) {
 printf("Correct! You win!\n");
 break;
 } else if (guess < secret) {
 printf("Too low! Try again.\n");
 } else {
 printf("Too high! Try again.\n");
 }
 }

 if (i == attempts && guess != secret) {
 printf("You lose! The number was %d.\n", secret);
 }
}

return 0;
}
```

(ChatGPT)

```

#include <stdio.h>

int main(void) {
 int secret = 7;
 int guess;
 int correct = 0;

 printf("Guess a number between 1 and 10.\n");

 for (int attempt = 1; attempt <= 3; attempt++) {
 printf("Attempt %d/3. Enter your guess: ", attempt);
 scanf("%d", &guess);

 if (guess == secret) {
 printf("Correct! You win!\n");
 correct = 1;
 break;
 } else if (guess < secret) {
 printf("Too low! Try again.\n");
 } else {
 printf("Too high! Try again.\n");
 }
 }

 if (!correct) {
 printf("You lose! The secret number was %d.\n", secret);
 }

 return 0;
}

```

2e.

**Correctness:** Both GenAIs performed the task with no trouble. However, both programs have edge case issues when the user inputs anything but a number. The program runs automatically without prompting the user, such as “Invalid input; please try again.” Thus, the scanf does not account for improper inputs.

**Execution Time:** Both GenAI’s use  $O(1)$  in practice. This means they both operate in constant time. Furthermore, both use a for loop instead of a while loop, which runs at most 3 iterations set by their attempt parameters. Thus, the code is roughly the same speed, with no notable difference other than how long it takes the user to enter a number on the prompt line.

**Space Complexity:** As stated before, both use  $O(1)$ , so neither uses extra data in that capacity. The notable differences, while small, are that ChatGPT uses an additional integer, correct. While this probably uses around four more bytes of storage, it’s not a significant difference in overall data usage between the two programs.

**Maintainability:** Claude has a noticeable maintainability difference compared to ChatGPT when examining how both approach announcing to the user that they lost. First, ChatGPT’s if statement for the user-losing scenario uses (!correct), which basically asks if correct is still 0. Additionally, it kept this statement outside the loop, which is more practical if the coder wants to change the type of loop they’re using. Now they don’t need to rewrite how they check whether the user won, and it prevents the logic from breaking if the coder wants to add

new features, like more hints. As for Claude, it's logic is a little more lengthly "(i == attempts && guess != secret)" with the code being inside the loop instead of on the outside. Therefore, the logic would have to be rewritten if the loop method was changed or if a new feature was added.

2f. I chose to go with ChatGPT based on the final results of comparing the two GenAI's. While both are similar in correctness, execution time, and space complexity the maintainability helped me make my final decision. ChatGPT having that leg up in terms of being easier to be maintain. For example, changing the type of loop or adding new features having it be able to not break down to where the logic needs to be rewritten. That is the primary reason why I went with it.

2G.

**Correctness:** For correctness I changed the scanf to account for non-numerical numbers. This allowed for the program to catch these errors and doesn't count them against someones guesses. Also, it now informs the user why the error occurred.

**Execution Time:** Nothing major changed in this aspect. However, for added the error check that did increase the process slightly with a few more operations, the program still remains as a constant  $O(1)$ , overall.

**Space Complexity:** I made no improvements to this aspect to keep the for loop consistent. Also, there was no major data strains in the first place with either programs so I didn't find it necessary. Thus, it remains as the constant  $O(1)$ .

**Maintainability:** I kept the if block that dealt with the if (!correct) on the outside. This was the factor I liked the most out of the two GenAIs the most. Furthermore, I added comments about what each line of code does in the program. This helps for later reference if someone want to know what is happening the program, so now anyone can maintain the code.